Fheanor: a new, modular FHE library for designing and optimising schemes

Hiroki Okada[2,3], Rachel Player[1], and Simon Pohmann[1]

[1] Royal Holloway, University of London, UK
[2] KDDI Research, Japan
[3] The University of Tokyo, Japan

Abstract. Implementations of modern FHE schemes are available in various highly-optimized libraries. Many of these libraries are designed to allow developers who may not have deep expertise in FHE to build fast and secure privacy-preserving applications. To support such users, the API of these libraries often hides the internals of the schemes in question from the user. However, this design choice makes it hard for users of these libraries to modify existing schemes, or implement new ones; work that is often valuable when conducting research on FHE schemes.
We present our new Rust library Fheanor, which aims to facilitate such research on FHE schemes. The core target user is an FHE researcher, rather than an application developer. Most importantly, the design of Fheanor is very modular, and mirrors the mathematical structure of the available FHE schemes. By exposing the mathematical structure, but still hiding implementation details, it is easy to modify or extend the functionality of FHE schemes implemented in the library and still preserve high performance. Indeed, Fheanor demonstrates performance that is close to that of HElib or SEAL, with the potential for optimizations in the future.
Fheanor implements several features that have not, or have only rarely, been implemented in previous libraries. These include non-power-of-two cyclotomic rings, single-RNS based ring arithmetic, the CLPX/GBFV scheme, and bootstrapping for BFV and BGV.
In addition, this paper presents new theoretical contributions that are also implemented in Fheanor. The first is an extension of optimal digit extraction circuits, used in BFV/BGV bootstrapping, to the case 2^{23}. The second is a more efficient algorithm for computing the trace in the non-power-of-two cyclotomic setting.

Keywords: Homomorphic Encryption, BFV, BGV, CLPX, GBFV, Implementation

1 Introduction

Fully Homomorphic Encryption (FHE) refers to encryption schemes that allow performing arbitrary computations on encrypted data, without requiring decryption. The first construction by Gentry [Gen09] was proposed more than 15 years

ago, and since then a variety of schemes have been developed, bringing FHE closer to practice. Broadly speaking, the currently studied schemes can be divided into two families. The first family is based on the BGV scheme [BGV12] and its scale-invariant counterpart BFV [FV12]. The CKKS scheme [CKKS17] and the CLPX/GBFV scheme [CLPX18; GV24] share most of the underlying techniques and structure of BGV and BFV, and can also be grouped in this family. The other family includes the DM/FHEW [DM15] and CGGI/TFHE [CGGI16] schemes.

All these schemes have been implemented in widely-used libraries, which have become quite fast due to a panoply of optimizations [HS14; 24; 23; ABB+22; Zam22b]. These optimizations include the ones published in [BEHZ16; HPS19; BEH+17], and many more that are 'folklore' or do not explicitly appear in the literature. Moreover, the HEIR [Con23] and Concrete [Zam22a] projects try to build compilers that convert non-FHE-aware code to a circuit that can be evaluated using FHE. As a result, application developers have now high-level access to fairly practical implementations of FHE schemes. However, application developers may not be the only users of these libraries: FHE researchers may also wish to implement and experiment with new ideas.

As existing implementations can be hard to modify, researchers that develop new schemes, or new variants of existing schemes, typically face the challenge of developing their own implementation. Moreover, researchers must include all the known optimizations in order to achieve competitive performance, which may be challenging if these optimizations are not documented. As a result, many interesting ideas in the literature are not implemented, or have only a slow, proof-of-concept implementation, like for example [CLPX18; GV23; BCH+25].

A very interesting, but not widely-implemented example of this is homomorphic encryption in non-power-of-two cyclotomic number rings. To the best of our knowledge, only $\Lambda \circ \lambda$ [CP16] and HElib [HS14] currently support these rings (see Table 1). This lack of implementations arguably is a serious obstacle for FHE experiments and optimizations in a non-power-of-two setting. For example [GV24] states that "due to the lack of non-power-of-two cyclotomics in state-of-the-art FHE libraries [supporting BFV]", they cannot benchmark their ideas in all applicable settings. More generally, we believe that non-power-of-two cyclotomics should receive more attention, because they allow for a higher number of slots with small plaintext moduli, thus enabling more SIMD parallelism in this parameter setting.

Another example is bootstrapping for BFV, which is intensively researched, but not implemented in any major library. Some isolated implementations have been published [CH18; GV23; OPP23], but none has been integrated into a major library. As a result, researchers who want to work on bootstrapping for BFV have to re-implement many components that are unrelated to the actual core of their work, even if they base their code on an existing HE library.

1.1 Our Contribution: Fheanor

We present our Rust library "Fheanor", which is available at https://github.com/FeanorTheElf/fheanor and https://crates.io/crates/fheanor. The library is pub-

lished under the MIT licence, and we are happy to include future contributions from the community. As opposed to previous libraries, Fheanor is primarily designed not for users who build applications leveraging FHE, but rather for FHE researchers who want to study and modify the internals of existing schemes.

Features The core idea underlying Fheanor is its very modular design, which follows the mathematical structure underlying homomorphic encryption schemes like BGV, BFV and CLPX/GBFV. Implementations of the fundamental rings $R/(q) = \mathbb{Z}[X]/(\Phi_m(X), q)$ and their arithmetic are in the center, as opposed to ciphertexts or "context objects", as in (e.g.) OpenFHE [ABB+22], HElib [HS14], or SEAL [23]. As a result of this design, implementing a new scheme in Fheanor is often as easy as writing down the corresponding formulas in code.

Within this paradigm, we implement the following features. These include some that have not, or have only rarely, been implemented in other libraries.

- **General cyclotomics.** Fheanor supports implementations over both power-of-two and non-power-of-two cyclotomic rings.
- **Double-RNS Ring Arithmetic.** Fheanor provides an implementation of fast ring arithmetic based on the "double-RNS" representation of ring elements.
- **Single-RNS Ring Arithmetic.** Fheanor contains also a second implementation of arithmetic in R_q based on a "single-RNS" representation. Single-RNS-based arithmetic can be faster in certain situations, so the option to easily change between double-RNS and single-RNS-based arithmetic gives additional flexibility.
- **HE schemes.** Fheanor contains implementations of the BFV, BGV and CLPX schemes. Note that while BGV has been implemented for non-power-of-two cyclotomic rings in HElib [HS14], no corresponding implementation of BFV is known to the authors, and CLPX is not implemented in any major library (c.f. Table 1).
- **Tools for Arithmetization.** Fheanor is the first HE library that explicitly models arithmetic circuits, and additionally provides methods to compute those (e.g. the Paterson-Stockmeyer method and HElib-style "BlockMatMul"), as well as novel arithmetizations for some common functions (including Trace, Norm, and digit extraction polynomials).
- **Bootstrapping for BFV/BGV.** Fheanor implements thin bootstrapping for both the BFV and BGV schemes. As far as we know, HElib [HS14] is the only library that supports BGV bootsrapping. Furthermore, while there have been implementations of BFV bootstrapping in the literature [CH18; GV23; OPP23], to the best of our knowledge none of them are part of any major library, nor are actively maintained.

Architecture An overview of the architecture of Fheanor is shown in Fig. 1. Fheanor builds on the `feanor-math` library[4], which provides an abstraction for

[4] Available at https://crates.io/crates/feanor-math.

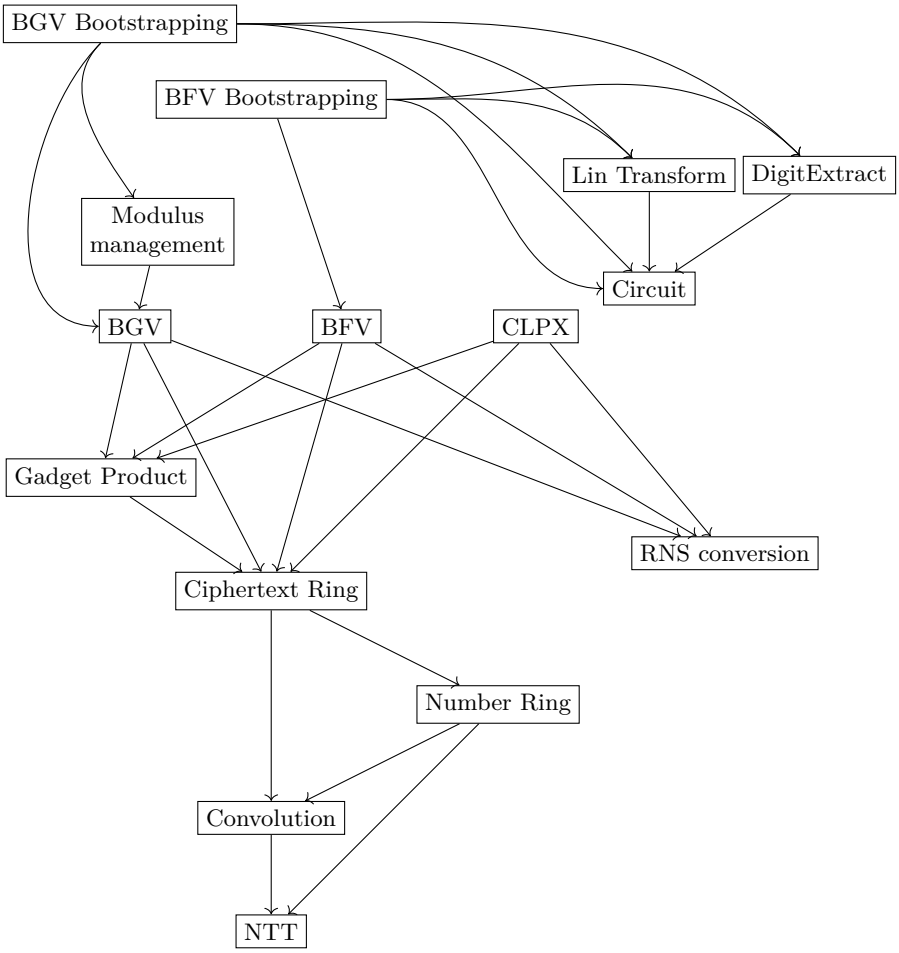

Fig. 1. Overview of the different modules of Fheanor. Arrows represent dependencies between the modules. Dependencies to the underlying mathematics library `feanor-math` are omitted for clarity.

rings and fundamental mathematical operations (that are not specific to HE). Furthermore, bindings to the Intel Homomorphic Encryption Acceleration (HEXL) library [BKS+21] are also available in a separate crate `feanor-math-hexl`, which is optionally used to provide a faster NTT.

In Figure 1, any layer or component is designed to be used individually, and so the library may be of interest to researchers in lattice-based cryptography more generally. For example, users that implement new lattice-based encryption schemes will find the "Number Ring", "Ciphertext Ring" and possibly the "RNS conversion" modules useful. Users that want to build a high-level application

leveraging FHE on the other hand are likely to primarily interact with the HE scheme modules such as "BGV" or "BFV", but can also manually access the underlying rings when necessary. Another example could be experimentation with different BGV modulus-switching strategies, which is possible since "BGV" is designed to be used without "Modulus management". Finally, it is also supported to instantiate higher-level components with custom implementations of dependencies, so one could for example experiment with novel ways of performing ring arithmetic by instantiating the BFV implementation with a custom ciphertext ring or NTT implementation.

Additional Contributions This paper presents additional theoretical contributions, that are also implemented in Fheanor, and that may be of independent interest.

First, we provide an improvement of the arithmetic circuits used for rounding elements of $\mathbb{Z}/p^e\mathbb{Z}$ during bootstrapping. This is done using "digit extraction polynomials", which realize the function $x \mapsto \text{shortest-lift}(x \bmod p) \bmod p^e$. In [GIKV23], the authors used Groebner basis techniques to find provably optimal arithmetic circuits computing the digit extraction polynomials modulo $2^2, 2^4, 2^8$ and 2^{16}. We extend this to provide an optimal circuit in the case of 2^{23}. We believe that this is a significant contribution, since many practically relevant parameter sets require a plaintext modulus between 2^{16} and 2^{23}. We adopted a novel approach using a SAT solver, as the increase in complexity meant that extending the Groebner basis approach of [GIKV23] was infeasible.

Second, we present a novel arithmetization of the field trace and field norm functions in non-power-of-two cyclotomics. For power-of-two cyclotomics, this technique is equal to the commonly known method based on intermediate fields [CH18; OPP23]. However, a generalization of the intermediate-fields method to non-power-of-two cyclotomics, as proposed by [CKL25], is no longer optimal in cases where the rank of the ring extension has moderately large prime factors. Instead, we propose a method based on short addition chains. This approach is optimal, assuming that the used addition chains are minimal.

1.2 Non-features

There are some features that are currently not implemented in Fheanor, either because they are planned for the future, or because they are intentionally considered out-of-scope. These are:

- **Variants of digit extraction**. There are many techniques in the literature [CH18; GIKV23; MHWW24a] that improve digit extraction in various specific settings. Currently, only some of these techniques ([HS15; CH18] and parts of [GIKV23]) are implemented. Fheanor supports the use of user-provided digit extraction polynomials or circuits, to encourage more research in this direction, and so it should be easy for users to incorporate other variants.

- **NTT optimizations.** We have already included various optimizations for computing NTTs and, more generally, arithmetic operations in R_q [Har14; BEHZ16]. However, there are still some known techniques that we plan to implement in the future. For example, we plan to implement truncated FFTs [Van04; Vrb16], which are used to compute NTTs in HElib.
- **Hardware acceleration.** We do not currently plan to include GPU or FPGA-based hardware acceleration in Fheanor. Users are of course welcome to implement these features, which are to a degree supported by our abstractions of NTTs and convolutions.
- **CKKS.** We plan to include an implementation of the CKKS scheme [CKKS17] in future, but it is not currently prioritized.
- **Noise estimation.** Fheanor does not currently include precise or rigorously justified noise estimation. Nevertheless, Fheanor defines an interface for noise estimators, and include a "naive" noise estimator based on asymptotic formulas [BGV12; FV12] for use during BGV bootstrapping. Improving this estimator to achieve the same quality as supported by other libraries (e.g. HElib [HS14] and OpenFHE [ABB+22]) is left to future work. As the target audience of Fheanor is researchers who are familiar with the internals of FHE, it is likely that users may wish to make the noise-related parameter decisions themselves, and so may not require an automated noise estimation tool.
- **Side-channel resistance.** We do not intend to implement side-channel protection techniques like masking, which would significantly complicate the underlying mathematical structure. Indeed, we believe exposing the mathematical structure in a direct and uncomplicated way is more useful for our target audience. To the best of our knowledge, no such side-channel protection measures have been implemented in other HE libraries either.

1.3 Related work

A comparison between Fheanor and other libraries implementing the BFV/BGV family of schemes is shown in Table 1. Of these libraries, $\Lambda \circ \lambda$ [CP16] is closest to Fheanor "in spirit", since the design of $\Lambda \circ \lambda$ also focuses on modelling the mathematical structure behind lattice-based cryptography. However, $\Lambda \circ \lambda$ is not built for FHE in particular, and thus its FHE-related feature set is limited: for example, bootstrapping or arithmetization tools are not implemented.

Comparing to libraries that primarily focus on FHE, Fheanor is closest to HElib [HS14]. Fheanor reuses many techniques that are already implemented in HElib, such as the hypercube structure for encoding multiple messages in the plaintext space, and the linear transforms used during bootstrapping. However, Fheanor also provides implementations of BFV and CLPX, which are not available in HElib.

1.4 Organization

The remainder of this paper is organized as follows. In Section 2, we introduce the necessary definitions and background. Then, in Section 3, we discuss the

Library	Language	BGV	BFV	CLPX	Bootstrap?	All m?
HElib [HS14]	C++	✓	✗	✗	✓	✓
OpenFHE [ABB+22]	C++	✓	✓	✗	✗	✗
Seal [23]	C++	✓	✓	✗	✗	✗
Lattigo [24]	Go	✓	✓	✗	✗	✗
$\Lambda \circ \lambda$ [CP16]	Haskell	✓	✓	✗	✗	✓
Fheanor	Rust	✓	✓	✓	✓	✓

Table 1. Overview over the capabilities of well-known FHE libraries, restricted to the BFV/BGV family of schemes. The column "Bootstrap?" marks which libraries implement bootstrapping (for BGV/BFV), and the column "All m?" refers to whether the scheme implementation supports general cyclotomic number rings (as opposed to only power-of-two cyclotomics).

implementation of ring arithmetic in $R_q = \mathbb{Z}[X]/(q, \Phi_m(X))$ in Fheanor for various m and q, and present benchmarks for various arithmetic circuits (without bootstrapping) that compare the performance of Fheanor with HElib and SEAL. In Section 4, we present our new algorithm for computing field trace and norm in non-power-of-two cyclotomic rings, and demonstrate its improvement over prior work. Finally, in Section 5, we give an overview of the implementations of BFV and BGV bootstrapping in Fheanor, and present our new optimal digit extraction circuits. We also present benchmarks for BGV thin bootstrapping that compare the performance of Fheanor with HElib.

2 Preliminaries and Background

2.1 Cyclotomic number rings

We let $R = \mathbb{Z}[\zeta_m]$ be the ring of integers in the m-th cyclotomic number field $\mathbb{Q}[\zeta_m]$. This ring has Galois group $\mathrm{Gal}(R/\mathbb{Z}) := \mathrm{Gal}(\mathbb{Q}[\zeta_m]/\mathbb{Q})$ isomorphic to $(\mathbb{Z}/m\mathbb{Z})^*$, given by

$$(\mathbb{Z}/m\mathbb{Z})^* \xrightarrow{\sim} \mathrm{Gal}(R/\mathbb{Z}), \quad k \mapsto \left(\sigma_k : \zeta_m \mapsto \zeta_m^k \right) .$$

We endow R with the *canonical norm*, given by

$$\|x\| = \|x\|_{\mathrm{can}} := \sqrt{\sum_{\sigma \in \mathrm{Gal}(R/\mathbb{Z})} \|\sigma(x)\|^2} .$$

When we say that elements of R are "small", we always refer to "small w.r.t. canonical norm".

If $m = m_1 \cdots m_r$ factors into pairwise coprime integers m_i, one can write R as a tensor product

$$R \cong \mathbb{Z}[\zeta_{m_1}] \otimes \ldots \otimes \mathbb{Z}[\zeta_{m_r}] \quad \text{via} \quad \zeta_{m_1}^{i_1} \otimes \ldots \otimes \zeta_{m_r}^{i_r} \mapsto \zeta_m^{i_1 m/m_1 + \ldots + i_r m/m_r} . \quad (1)$$

This can be used to define the powerful basis [LPR13], which consists of $\zeta_{m_1}^{i_1} \otimes ... \otimes \zeta_{m_r}^{i_r}$ for $i_j \in \{0, ..., \phi(m_j) - 1\}$. Note that the powerful basis (as well as the coefficient-basis ζ_m^i for $i \in \{0, ..., \phi(m) - 1\}$) consists of small elements w.r.t. the canonical norm, concretely $\|\zeta_{m_1}^{i_1} \otimes ... \otimes \zeta_{m_r}^{i_r}\| = \sqrt{\phi(m)}$.

Next, we turn our attention to the quotient of R by an integer q. If $q = p$ is a prime, it is well-known that

$$R_p \cong \mathbb{F}_{p^d}^{\phi(m)/d}$$

decomposes into a direct product of $\phi(m)/d$ copies of $\mathbb{F}_{p^d}$, where $d = \mathrm{ord}_{(\mathbb{Z}/m\mathbb{Z})^*}(p) = |\langle p \rangle|$ is the order of p in $(\mathbb{Z}/m\mathbb{Z})^*$, or equivalently the size of the subgroup $\langle p \rangle$. Another way to phrase this, which preserves more information about the Galois group, is to pick any prime ideal $\mathfrak{p}_0$ of R over p and any set of representatives S for $(\mathbb{Z}/m\mathbb{Z})^*/\langle p \rangle$, and note that we have the isomorphism

$$R_p \xrightarrow{\sim} \mathbb{F}_{p^d}^S, \quad x \mapsto (\sigma_k(x) \bmod \mathfrak{p}_0)_{k \in S}\,.$$

The situation becomes much simpler if we restrict to $d = 1$, i.e. $p \equiv 1 \bmod m$. In this case, $R_p \cong \mathbb{F}_p^{\phi(m)}$, and we can extend this to any modulus $q = p_1 \cdots p_l$ for which every $p_i \equiv 1 \bmod m$. We then get

$$R_q \cong \bigoplus_{i=1}^{l} \bigoplus_{j=1}^{\phi(m)} \mathbb{F}_{p_i} \tag{2}$$

This decomposition is called the *double-RNS representation* of R_p, and is very useful for implementations, since addition and multiplication of ring elements is just component-wise addition resp. multiplication of elements of $\mathbb{F}_{p_i}$.

2.2 The Number-Theoretic Transform

The Number-Theoretic Transform (NTT) refers to the Fourier Transform over a finite field $\mathbb{F}_p$. Hence, the length-m NTT refers to the map

$$\mathrm{NTT}_m : \mathbb{F}_p^m \to \mathbb{F}_p^m, \quad (a_i)_{0 \le i < m} \mapsto \Big(\sum_{j=0}^{m-1} \xi^{ij} a_j\Big)$$

for a primitive m-th root of unity $\xi \in \mathbb{F}_p$.

The importance of the NTT is twofold. On the one hand, it can be used to explicitly compute the isomorphism Equation (2). Indeed, if p splits completely in $\mathbb{Z}[\zeta_m]$ into prime ideals $\mathfrak{p}_1, ..., \mathfrak{p}_{\phi(m)}$, it holds that $\mathfrak{p}_i = (p, \zeta_m - \xi^{k_i})$ for some $k_i \in (\mathbb{Z}/m\mathbb{Z})^*$. Hence, the reduction modulo $\mathfrak{p}_i$ is equivalent to the evaluation at ξ^{k_i}, and thus ignoring the components $(\mathbb{Z}/m\mathbb{Z}) \setminus (\mathbb{Z}/m\mathbb{Z})^*$ in the NTT output gives the isomorphism

$$R_p \xrightarrow{\sim} \mathbb{F}_p^{(\mathbb{Z}/m\mathbb{Z})^*}, \quad x \mapsto (\mathrm{NTT}(x)_k)_{k \in (\mathbb{Z}/m\mathbb{Z})^*}\,.$$

If m is a power of two, it is possible to modify the NTT to only compute the required indices $\in (\mathbb{Z}/m\mathbb{Z})^*$, but except for this case, the we are not aware of a method that avoids computing the NTT coefficients $\in (\mathbb{Z}/m\mathbb{Z}) \setminus (\mathbb{Z}/m\mathbb{Z})^*$.

On the other hand, the NTT can also be used to compute arbitrary convolutions, via the convolution theorem. In particular, if $a(X) = \sum_i a_i X^i, a'(X) = \sum_i a'_i X^i \in \mathbb{F}_p[X]$ are polynomials of joint degree $\deg(a) + \deg(a') < m$, then the coefficients of aa' are given by

$$\mathrm{NTT}_m^{-1}(\mathrm{NTT}_m((a_i)_i) \odot \mathrm{NTT}_m((a'_i)_i)),$$

where $\odot$ refers to the coefficient-wise product of vectors.

It is well-known that the Fourier Transform (and similarly the NTT) can be computed in time $O(m \log(m))$: this is commonly referred to as Fast Fourier Transform (FFT), which gives the fastest known algorithm for polynomial multiplication over $\mathbb{F}_p$ for suitable p. If m is a power of two, many very efficient FFT algorithms are known, with the Cooley–Tukey FFT being the most commonly used one. Therefore, the Cooley–Tukey FFT algorithm has become a fundamental tool in HE and lattice-based cryptography, and various optimizations for this specific case have been proposed [Har14; LN16; Sco17].

If m is not a smooth integer (say, a prime), the situation becomes much more complicated. The most common FFT algorithm in this setting is the Bluestein FFT [Blu70], but internally it performs multiple FFTs of power-of-two length $\geq 2n$, and is thus much slower in practice.

2.3 The BGV/BFV family of HE schemes

The most widely-used FHE schemes (including all that are currently being standardized) are based on the Ring Learning with Errors (RLWE) assumption. RLWE was proposed by [LPR10; SSTX09] as a structured variant of LWE [Reg05], and has since become one of the most important foundations for modern cryptography.

Definition 1 (RLWE problem). For m, q, and a distribution χ over R_q, the problem $\mathrm{RLWE}(m, q, \chi)$ is the problem of distinguishing the probability ensemble

$$(a, as + \epsilon),\ a \xleftarrow{\$} R_q,\ \epsilon \leftarrow \chi$$

from uniform, where s is a fixed element of R_q.

It is conjectured that, for suitable parameters, the RLWE problem is very hard to solve computationally. Hence, we can use an RLWE sample (a, b) to (symmetrically) encrypt a ring element μ as $(a, \mu + b)$. This is the basis of most lattice-based encryption schemes, including FHE schemes. An important consequence is that we include the noise term ϵ in every encryption. Hence, schemes must encode the message μ in a way that allows removal of this noise term during decryption. The main difference between BGV and BFV is how this encoding works. While BGV encrypts the message in the lowest bits of a ciphertext, BFV encrypts the message in the highest bits.

BFV For simplicity, we present BFV first.

Definition 2 (The BFV scheme). For a cyclotomic ring R and integers q (called *ciphertext modulus*) and t (called *plaintext modulus*), we define the BFV encryption scheme as follows. Note that the rounding $\lfloor \cdot \rceil$ should be applied coefficient-wise.

- KeyGen(): Return a random $s \in R_q$ with coefficients in $\{-1, 0, 1\}$.
- Enc($s, \mu \in R_t$): Generate an RLWE sample (a, b) using s and return $(a, \lfloor q\mu/t \rfloor - b) \in R_q^2$.
- Dec($s, (c_1, c_0) \in R_q^2$): Return $\lfloor t(c_0 + c_1 s)/q \rceil \in R_t$.
- HomAdd($(c_1, c_0), (c_1', c_0')$): Return $(c_1 + c_1', c_0 + c_0')$.
- HomMul($(c_1, c_0), (c_1', c_0'), \mathrm{rk}$): Compute the intermediate "3-part ciphertext"

$$(c_2'', c_1'', c_0'') := \left(\left\lfloor \frac{t}{q} \hat{c}_1 \hat{c}_1' \right\rceil, \ \left\lfloor \frac{t}{q} (\hat{c}_0 \hat{c}_1' + \hat{c}_1 \hat{c}_0') \right\rceil, \ \left\lfloor \frac{t}{q} \hat{c}_0 \hat{c}_0' \right\rceil \right) \in R_q^3,$$

  where $\hat{c}_i$ and $\hat{c}_i'$ are the shortest lifts of c_i resp. c_i' to R. Afterwards, apply relinearization to (c_2'', c_1'', c_0'') and return the result, i.e. Relin$((c_2'', c_1'', c_0''), \mathrm{rk})$.
- RelinKeyGen(s): Choose a so-called "gadget vector" $g \in \mathbb{Z}^l$, which should allow decomposing any number $x \in \mathbb{Z}$ as a linear combination $x = \sum_i g_i x_i$ with small $x_i \in \mathbb{Z}$. Then generate RLWE samples $(a_1, b_1), ..., (a_l, b_l)$ and return

$$\mathrm{rk} = ((a_1, g_1 s^2 - b_1), ..., (a_l, g_l s^2 - b_l))$$

- Relin($(c_2, c_1, c_0), \mathrm{rk} = ((k_{11}, k_{10}), ..., (k_{l1}, k_{l0}))$): Using the same gadget vector g as for RelinKeyGen, decompose c_2 as $c_2 = \sum_i g_i x_i$ with small ring elements $x_i \in R_q$. Return

$$(c_1', c_0') := \left(c_1 + \sum_i k_{i1} x_i, \ c_0 + \sum_i k_{i0} x_i \right)$$

For a discussion of the correctness and security of this scheme, we refer to the original work [FV12].

Finally, another important operation in BFV is the so-called modulus-switching. Given a ciphertext $(c_1, c_0) \in R_q$ and another integer q', we can compute $(\lfloor q' c_1/q \rceil, \lfloor q' c_0/q \rceil) \in R_{q'}^2$, which is a valid BFV ciphertext encrypting the same message w.r.t. ciphertext modulus q'. It can for example be used to reduce the size of ciphertexts before transmission, and is usually applied before bootstrapping to map a ciphertext into the plaintext space of the bootstrapping scheme.

BGV Next, we summarize the BGV scheme [BGV12]. The main difference is that instead of scaling the message by q/t as in BFV, in BGV we leave the message unchanged, but scale the error by t. Then, instead of rounding, we can retrieve the message using reduction by t.

Definition 3 (The BGV scheme). For a cyclotomic ring R and integers q (called *ciphertext modulus*) and t (called *plaintext modulus*), we define the BGV encryption scheme as follows

- KeyGen(): Return a random $s \in R_q$ with coefficients in $\{-1, 0, 1\}$.
- Enc$(s, \mu \in R_t)$: Generate an RLWE sample (a, b) using s and return $(at, \mu - bt) \in R_q^2$.
- Dec$(s, (c_1, c_0) \in R_q^2)$: Return shortest-lift$(c_0 + c_1 s) \bmod t \in R_t$.
- HomAdd$((c_1, c_0), (c_1', c_0'))$: Return $(c_1 + c_1', c_0 + c_0')$.
- HomMul$((c_1, c_0), (c_1', c_0'), \text{rk})$: Compute the intermediate "3-part ciphertext"

$$(c_2'', c_1'', c_0'') := (c_1 c_1', c_0 c_1' + c_1 c_0', c_0 c_0') \in R_q^3 .$$

  Then apply relinearization to (c_2'', c_1'', c_0'') and return the result Relin$((c_2'', c_1'', c_0''), \text{rk})$.
- RelinKeyGen(s): Same as for BFV (see Definition 2).
- Relin$((c_2, c_1, c_0), \text{rk} = ((k_{11}, k_{10}), ..., (k_{l1}, k_{l0})))$: Same as for BFV (see Definition 2).
- ModSwitch$((c_1, c_0), q, q')$: Return (c_1', c_0') where $c_i' \in R_{q'}$ is the closest ring element to $q' c_i / q$ that satisfies $q c_i' = q' c_i \bmod t$.

It seems like multiplication is now much simpler than in the BFV case, since for $c_0 + c_1 s = \mu + t\epsilon$ and $c_0' + c_1' s = \mu' + t\epsilon'$, the triple $(c_1 c_1', c_0 c_1' + c_1 c_0', c_0 c_0')$ already satisfies

$$c_1 c_1' s^2 + (c_0 c_1' + c_1 c_0') s + c_0 c_0' = (c_1 s + c_0)(c_1' s + c_0') = \mu\mu' + t(\epsilon\mu' + \epsilon'\mu) + t^2 \epsilon\epsilon'.$$

However, the product $\epsilon\epsilon'$ means that the growth in this case is multiplicative, i.e. the *relative* noise in the result (relative w.r.t. q) increases by a factor proportional to the *absolute* noise of the input ciphertexts, instead of a constant factor as in BFV. This was solved by [BGV12] by introducing modulus-switching, since switching from q to a smaller ciphertext modulus q' decreases the absolute size of the noise by a factor of q/q', while keeping the relative noise constant. Therefore, modulus-switching in BGV plays a much more important role than in BFV, since we need to perform it regularly before homomorphic multiplications to achieve asymptotically the same noise growth as in BFV.

This is also one of the main inconveniences of BGV: instead of having a fixed ciphertext space R_q, the ciphertext modulus of BGV must constantly decrease (via modulus-switching) during homomorphic computations, in order to limit the noise growth. Hence, it is necessary to manage a list of ciphertext modulus $q_0 > q_1 > q_2 > ...$ and the corresponding ciphertext rings R_{q_i}. This is not necessary for BFV, but may be used as a performance optimization. On the other hand, BGV avoids the inconvenient computation of lifts $\hat{c}_i, \hat{c}_i'$ of elements of R_q that is required during BFV multiplication.

CLPX/GBFV The third scheme that is currently implemented in Fheanor is CLPX/GBFV [CLPX18; GV24], which can be seen as variant of BFV. The motivation behind CLPX is to change the plaintext ring from R_t to some ring that directly supports computations with large integers. This is necessary, since the noise growth of BFV/BGV scales linearly with t, and thus in practice t is

chosen no larger than 16-bit. However, this clearly makes representing larger integers as elements of R_t difficult.

CLPX solves this problem by removing the restriction that t be an integer, and replacing it with an arbitrary element of R. In the original paper [CLPX18], the authors chose $t = \zeta_m - 2$, in which case one finds that $R/(t) \cong \mathbb{Z}/\Phi_m(2)\mathbb{Z}$ is a ring of very large characteristic. Perhaps surprisingly, it turns out that BFV works with general $t \in R$, almost without further adjustments. In [GV24], the authors have explored different choices of t that make $R/(t)$ isomorphic to a product of $\mathbb{Z}/p\mathbb{Z}$ for various p, of size between 16 and 64 bits.

2.4 Slots and Galois automorphisms

We have already seen that for a prime p, the plaintext space of BFV/BGV allows for a decomposition into "slots"

$$R_p \cong \mathbb{F}_{p^d}^S$$

for a set of representatives S of $(\mathbb{Z}/m\mathbb{Z})^*/\langle p \rangle$. This generalizes to the case that the plaintext modulus $t = p^e$ is a prime power, in which case we have $R_t \cong \mathrm{GR}(p, e, d)^S$, where $\mathrm{GR}(p, e, d)$ denotes the rank-d Galois ring of characteristic p^e.

This is a fundamental reason why BFV/BGV is so successful: if we choose suitable parameters, this allows for a kind of SIMD parallelism. More concretely, it allows us to "pack" $\phi(m)/d$ elements of $\mathbb{F}_{p^d}$ (resp. $\mathrm{GR}(p, e, d)$) into a single plaintext, and encrypt that packed plaintext. Now arithmetic operations on packed plaintexts are performed slot-wise, e.g. the homomorphic multiplication of two ciphertexts corresponds to $\phi(m)/d$ multiplications of encrypted slot elements.

Using a technique similiar to relinearization, it is possible to generate "Galois keys" and use them to homomorphically compute the action of a Galois automorphism $\sigma \in \mathrm{Gal}(R/\mathbb{Z})$ on encrypted messages. This can be used to move data between these slots, since we have

$$\sigma_k : \mathbb{F}_{p^d}^S \to \mathbb{F}_{p^d}^S, \quad (a_i)_{i \in S} \mapsto (\pi^{l(i)}(a_{j(i)}))_{i \in S} \quad \text{where } j(i)k = ip^{l(i)}\,.$$

where $\pi : \mathbb{F}_{p^d} \to \mathbb{F}_{p^d}$ is the power-of-p Frobenius automorphism. To simplify this, [HS20] proposed to choose a map

$$h : \{0, ..., m_1 - 1\} \times ... \times \{0, ..., m_r - 1\} \to (\mathbb{Z}/m\mathbb{Z})^*, \quad (i_1, ..., i_r) \mapsto \prod_j g_j^{i_j}$$

that is a bijection "modulo $\langle p \rangle$", i.e. the operation $(\cdot \bmod \langle p \rangle) \circ h$ is bijective. We call such maps "hypercube structures" on $(\mathbb{Z}/m\mathbb{Z})^*$ modulo $\langle p \rangle$, as they "arrange" the elements of $S := \mathrm{im}(h)$, which serve as indices for the slots, along an r-dimensional hypercube $m_1 \times ... \times m_r$. Furthermore, the Galois automorphism $\sigma_{h(0,...,\delta,...,0)}$ then "shifts" the content of each slot by δ positions along the j-th dimension. Formally, we define the "slot unit vectors"

$$u_{i_1,...,i_r} := \left(\begin{cases} 1 & \text{if } s = h(i_1, ..., i_r) \\ 0 & \text{otherwise} \end{cases} \right)_{s \in S} \in \mathbb{F}_{p^d}^S \cong R_p, \quad i_1 < m_1, ..., i_r < m_r$$

and observe that for $i_j + \delta < m_j$, it holds that

$$\sigma_{h(0,\ldots,\delta,\ldots,0)}(u_{i_1,\ldots,i_r}) = u_{i_1,\ldots,i_j+\delta,\ldots,i_r} .$$

While different hypercube structures have been proposed [CCLS19; HS20], the most important one is the one used in HElib, since it respects the tensor decomposition

$$\mathbb{Z}[\zeta_m] = \mathbb{Z}[\zeta_{m_1}] \otimes \ldots \otimes \mathbb{Z}[\zeta_{m_r}] ,$$

where $m = m_1 \cdots m_r$ is the factorization of m into coprime prime powers. We make the restriction that m is odd, in which case every $(\mathbb{Z}/m_i\mathbb{Z})^*$ is cyclic. Thus, we can choose $g_i \in (\mathbb{Z}/m\mathbb{Z})^*$ such that $g_i \bmod m_i$ is a generator of $(\mathbb{Z}/m_i\mathbb{Z})^*$ and $g_i \equiv 1 \bmod m_j$ for $j \neq i$. Furthermore, we choose

$$m_i := \frac{\mathrm{ord}_{(\mathbb{Z}/m_1\cdots m_{i-1}\mathbb{Z})^*}(p)\phi(m_i)}{\mathrm{ord}_{(\mathbb{Z}/m_1\cdots m_i\mathbb{Z})^*}(p)} .$$

We call this specific hypercube structure the "Halevi-Shoup hypercube", and it allows a more efficient Coeffs-to-Slots transform during BFV and BGV bootstrapping.

2.5 Bootstrapping

As mentioned previously, FHE ciphertexts contain noise, which increases during homomorphic operations. Hence, to perform computations of arbitrary length, it is necessary to decrease this noise again. The only known way to do this is the celebrated bootstrapping technique by Gentry [Gen09]. The underlying idea is quite simple: given a high-noise ciphertext, we homomorphically run the decryption algorithm, using an encryption of the secret key.

The modern bootstrapping algorithm for BFV and BGV has been developed in a series of works [HS15; CH18; GIKV23; OPP23; MHWW24a; Gee24; MHWW24b]. Its basic structure has remained the same, and is displayed in Fig. 2. The main challenge in bootstrapping is the rounding operation, since representing it via arithmetic operations is quite expensive. The best known approach to do this is to work in prime-power characteristic p^e, and consider "digit extraction polynomials", which represent the map

$$\mathbb{Z}/p^e\mathbb{Z} \to \mathbb{Z}/p^e\mathbb{Z}, \quad x \mapsto \text{shortest-lift}(x \bmod p) \bmod p^e .$$

That is, we "extract" the lowest digit in the p-adic representation of x. By repeatedly using digit extraction to extract the lowest digit, and performing exact division by p (which is supported by BFV and BGV), we can implement flooring, or equivalently rounding.

However, naively applying this digit extraction procedure will only recover a single coefficient of the result. In order to avoid running the expensive digit extraction procedure $\phi(m)$ times, the SIMD parallelism provided by the slots is used. More concretely, using Galois automorphisms, we apply a linear transform

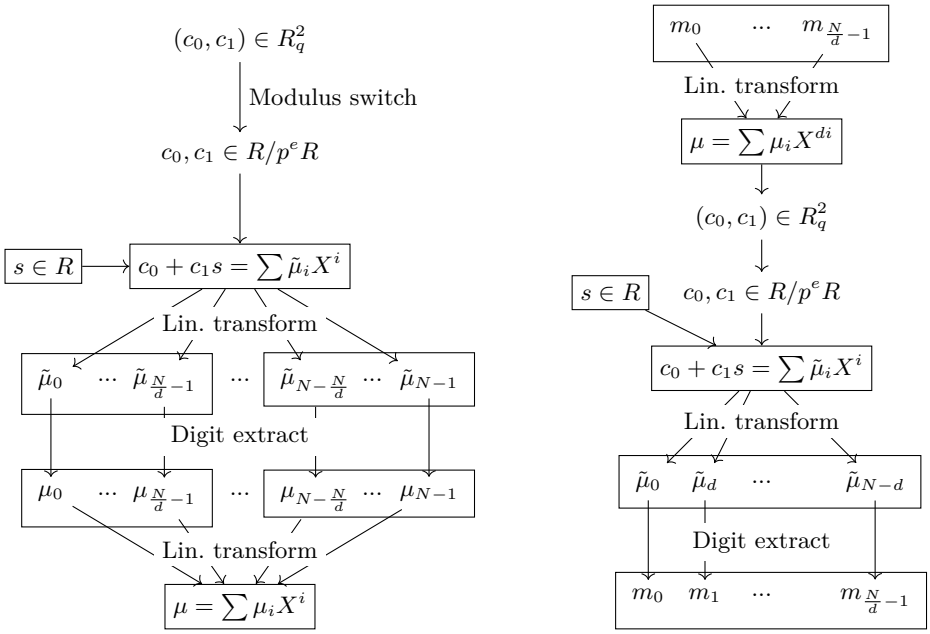

Fig. 2. The abstract bootstrapping procedure for BFV and BGV using slots, with "fat" bootstrapping on the left and "thin" bootstrapping on the right. Each rectangle represents one ciphertext. Figure is taken from [OPP23], and uses the notation $N = \phi(m)$.

(the Coeffs-to-Slots transform[5]) that moves the coefficients of the "noisy message" $p^e\mu/t + \epsilon$ into the slots, which means the digit extraction procedure will be applied to every coefficient at once. In other words, we only have to run digit extraction in total $d = \phi(m)/(\phi(m)/d)$ times, where $\phi(m)/d$ is the number of slots.

In the case that the input ciphertext only contains a single scalar (i.e. element of $\mathbb{Z}/t\mathbb{Z}$) in each slot, we can do even better using "thin bootstrapping". By applying the linear transform before the inner product step, we can achieve that only $\phi(m)/d$ coefficients are of relevance and have to be retrieved. In this case, we only require a single execution of the digit extraction procedure.

3 The ciphertext ring implementation(s) in Fheanor

Since the performance of HE schemes is almost completely determined by the arithmetic in the ciphertext ring R_q, providing an efficient implementation of this

[5] We remark that "PowCoeffs-to-Slots" might be a better name, since the transform (as proposed in HElib) actually moves the coefficients w.r.t. the powerful basis into the slots. When using the Halevi-Shoup-hypercube, this transform can be performed faster than moving the coefficient-basis coefficients into the slots.

ring is one of the core contributions of Fheanor. In the case where m is a power of two, it is well-established that the double-RNS representation of ring elements is the most efficient. In the case of general cyclotomics, the most efficient approach is less clear. The reason is that we cannot keep elements stored only in double-RNS representation, since some operations (in particular the short lifts during BFV homomorphic multiplication and relinearization) require a representation w.r.t. a "small" basis (i.e. consisting of small ring elements), for example the coefficient basis. However, converting between these bases is quite expensive when m is not a power of two, since it requires the use of a non-power-of-two NTT (usually the Bluestein FFT [Blu70]), and possibly the polynomial reduction by Φ_m. Hence, to simplify and encourage experimentation in this setting, we provide another ring implementation based on the single-RNS representation.

In addition to these two implementations of R_q, our library supports any custom ring implementation that provides the necessary operations (defined by the trait `BGFVCiphertextRing`). Note that all supported rings currently must be based on an RNS representation of q as $q = p_1 \cdots p_r$, which can however be trivial, i.e. $q = p_1$. Usually the p_i are primes, but this is not necessary—the only restrictions are that they are pairwise coprime, and each is somewhat smaller than 64 bits.

3.1 Using the Double-RNS representation

Our first implementation (given by `DoubleRNSRing` and `ManagedDoubleRNSRing`, where the latter automatically switches between small-basis and double-RNS representation) stores elements by default using the double-RNS representation. For operations that require a small basis, as opposed to previous implementations [23; HS14], we decide to use the powerful basis. As long as m is prime power (in particular this includes the power-of-two case), this is equivalent to using the coefficient basis, but as soon as we have a nontrivial tensor decomposition

$$R = \mathbb{Z}[\zeta_{m_1}] \otimes ... \otimes \mathbb{Z}[\zeta_{m_r}]$$

this allows for a more efficient implementation. More concretely, it allows replacing a large Bluestein NTT with multiple smaller ones, and avoids the reduction modulo Φ_m during the reverse transform. The reason is that the powerful-basis-to-NTT-basis transform is the linear tensor product of the corresponding transforms for each m_i. Formally, if we define the transform as

$$f_m : \mathbb{Z}[\zeta_m]/(p) \to \mathbb{F}_p^{(\mathbb{Z}/m\mathbb{Z})^*}, \quad x \mapsto (x \bmod (\zeta - \xi_m^k))_{k \in (\mathbb{Z}/m\mathbb{Z})^*}$$

we find that

$$f_m = f_{m_1} \otimes ... \otimes f_{m_r} \,,$$

where $\xi_{m_i} \in \mathbb{F}_p$ are primitive m_i-th roots of unity in $\mathbb{F}_p$ and $\xi_m = \xi_{m_1} \otimes ... \otimes \xi_{m_r}$. Since the powerful basis is also given by the tensor product as $\zeta_{m_1}^{i_1} \otimes ... \otimes \zeta_{m_r}^{i_r}$, we can compute every $\mathrm{id} \otimes ... \otimes f_{m_i} \otimes ... \otimes \mathrm{id}$ by operating separately on the coefficients w.r.t. every group of powerful basis vectors

$$\zeta_{m_1}^{i_1} \otimes ... \otimes 1 \otimes ... \otimes \zeta_{m_r}^{i_r}, \quad ..., \quad \zeta_{m_1}^{i_1} \otimes ... \otimes \zeta_{m_i}^{\phi(m_i)-1} \otimes ... \otimes \zeta_{m_r}^{i_r} \,.$$

Algorithm 1: Powerful-basis-to-NTT-basis conversion algorithm

Input: Powerful-basis coefficients $a_{i_1,\ldots,i_r}$ of
$a = \sum_i a_{i_1,\ldots,i_r} \zeta_{m_1}^{i_1} \otimes \ldots \otimes \zeta_{m_r}^{i_r} \in R_p$

Output: NTT-basis coefficients $a_k = a \bmod (\zeta_m - \xi^k) \in \mathbb{F}_p$ for $k \in (\mathbb{Z}/m\mathbb{Z})^*$, with a fixed m-th root of unity $\xi \in \mathbb{F}_p$

Set $(a^{(0)}_{1,i_1,\ldots,i_r})_{i_1,\ldots,i_r}$ to $(a_{i_1,\ldots,i_r})_{i_1,\ldots,i_r}$
for $j \in \{1, \ldots, r\}$ **do**
  for $(i_{j+1}, \ldots, i_r) \in \{0, \ldots, \phi(m_{j+1}) - 1\} \times \ldots \times \{0, \ldots, \phi(m_r) - 1\}$ **do**
    for $k \in (\mathbb{Z}/m_1 \cdots m_{j-1}\mathbb{Z})^*$ **do**
      Set $(f_l)_{l \in \mathbb{Z}/m_j\mathbb{Z}}$ to be the length-m_j NTT of $(a^{(j-1)}_{k,i_j,\ldots,i_r})_{0 \le i_j < \phi(m_j)}$ (zero-padded to length m_j)
      Set $(a^{(j)}_{\mathrm{InvCRT}(k,l),i_{j+1},\ldots,i_r})_{l \in (\mathbb{Z}/m_j\mathbb{Z})^*}$ to $(f_l)_{l \in (\mathbb{Z}/m_j\mathbb{Z})^*}$
return $(a^{(r)}_k)_{k \in (\mathbb{Z}/m\mathbb{Z})^*}$

Chaining these transforms for every i, we arrive at Algorithm 1. We note that this property of the powerful basis is well-known and has already been used by [CP16] for the from-and to-NTT-basis transforms. To the best of our knowledge, [HS14] however does not use it for ciphertext arithmetic, but it was used in the design of the Slots-to-Coeffs transform used during bootstrapping [HS15]. Note that this optimization is not relevant for other libraries, since it doesn't apply to the power-of-two case.

3.2 Using the Single-RNS representation

Despite these optimizations, the conversion between powerful-basis representation and NTT-basis representation remains expensive. In particular, when m is not a power of two, we cannot avoid the use of the Bluestein FFT, which internally performs multiple power-of-two FFTs. However, we require the NTT-basis representation only for two cases: when multiplying ring elements, and when we want to compute the image under a Galois automorphism.

It turns out that in the non-power-of-two case, there is a faster way of computing a single multiplication of ring elements. In particular, the well-known convolution-theorem-based polynomial multiplication method requires fewer power-of-two NTTs than the conversion to double-RNS form. This means we can multiply two ring elements $a, a' \in R_p$ by interpreting them as polynomials $a(X), a'(X)$ of degree $< \phi(m)$ (with $a(\zeta_m) = a$, $a'(\zeta_m) = a'$), and reduce the resulting polynomial $a(X)a'(X)$ modulo $\Phi_m(X)$. Fast methods for this reduction based on Barett-reduction have previously been proposed, e.g. in [BEH+17], but in most cases a complete reduction is not necessary, and it suffices to reduce the product modulo $X^m - 1$ only.

We remark that this technique requires NTTs for every multiplication, while using the double-RNS representation means that all multiplications are very cheap after an expensive, initial representation conversion. However, in HE schemes, it

is common to alternate between multiplications and operations that require a small basis representation, hence it seems likely that in these cases, a single-RNS representation can yield performance advantages.

We implemented this algorithm in `SingleRNSRing`. Note that we still store each coefficient modulo each prime factor p of q separately, since this avoids large integer arithmetic. Hence, we call this the "single-RNS representation".

3.3 RNS conversions

During modulus-switching and homomorphic multiplication in BFV, we need to implement the rounded-rescaling map

$$\mathbb{Z}_q \to \mathbb{Z}_{q'}, \quad x \mapsto \lfloor q'x/q \rceil$$

(implemented in `AlmostExactRescale`), and change-of-modulus map

$$\mathbb{Z}_q \to \mathbb{Z}_{q'}, \quad x \mapsto \text{shortest-lift}(x) \bmod q'$$

(implemented in `AlmostExactBaseConversion`). For efficiency reasons, it is crucial that these are implemented without the use of arbitrary-precision integers, but instead directly on the RNS representation of $x \in \mathbb{Z}_q$, i.e. working with $x \bmod p_1, ..., x \bmod p_r$. In [BEHZ16], algorithms that achieve this have been proposed, except that they can introduce an error of ± 1 (this is why our implementations are called `AlmostExact*`). This additional error is not a problem, since it is much smaller than the noise term. We implement the algorithms of [BEHZ16] with a minor adaption: we compute the correction term using fixed-precision floating point numbers, instead of using arithmetic modulo some "helper modulus".

We also provide the rescaling variant that is required for BGV modulus switching (in `CongruencePreservingRescaling`). Concretely, this is the function that maps x to the closest integer to $q'x/q$ that is congruent to $q'q^{-1}x$ modulo t, i.e.

$$\mathbb{Z}_q \to \mathbb{Z}_{q'}, \quad x \mapsto \lfloor q'x/q \rceil + \text{shortest-lift}(q'q^{-1}x - \lfloor q'x/q \rceil \bmod t)\,.$$

3.4 Benchmarks

To compare the performance of Fheanor with other libraries, we perform benchmarks. We focus our benchmarks on the homomorphic multiplication operation, which is usually the most expensive part of many HE evaluations, and involves basically all components of the library. All benchmarks are single-threaded, and run on a system with an Intel Xeon E-2246G CPU and 64 GB of RAM. We used the version 0.7.2 version of Fheanor[6], with the Intel HEXL library [BKS+21] enabled for computations in power-of-two cyclotomic number rings. As our CPU does not support AVX-512 instructions we used only the unvectorized NTT from HEXL (nevertheless, this is faster than our native pure-Rust NTT). We will make available our code so that these benchmarks can be reproduced.

$(m, t, \log_2(q), c)$	Benchmark	HElib	Fheanor
$(85 \cdot 257, 7, 430, 4)$	Tree-product, 15 muls	301 ms	410 ms
$(85 \cdot 257, 7, 430, 4)$	Repeated squaring, 4 muls	252 ms	339 ms
$(2^{16}, 7, 865, 4)$	Tree-product, 15 muls	486 ms	244 ms
$(2^{16}, 7, 865, 4)$	Repeated squaring, 4 muls	399 ms	212 ms
$(337 \cdot 127, 7, 1120, 4)$	Tree-product, 15 muls	1409 ms	1818 ms
$(337 \cdot 127, 7, 1120, 4)$	Repeated squaring, 4 muls	1205 ms	1476 ms

Table 2. Comparison of timings for BGV homomorphic multiplication. The two benchmarks are the computation of the product of many inputs using a tree-reduction, and the repeated squaring of a single input ciphertext, respectively. The given multiplication time is the average time spent per homomorphic multiplication (including modulus-switching). The parameter c is the length of the gadget vector used for relinearization, and t is the plaintext modulus.

$(m, t, \log_2(q), c)$	Benchmark	SEAL		Fheanor
		`USE_INTRIN=ON`	`OFF`	
$(2^{15}, 5, 430, 3)$	Tree-product, 15 muls	76 ms	154 ms	216 ms
$(2^{15}, 5, 430, 3)$	Repeated squaring, 4 muls	60 ms	124 ms	214 ms
$(2^{16}, 5, 865, 3)$	Tree-product, 15 muls	449 ms	915 ms	1057 ms
$(2^{16}, 5, 865, 3)$	Repeated squaring, 4 muls	371 ms	761 ms	1015 ms

Table 3. Comparison of timings for BFV homomorphic multiplication. The two benchmarks are the computation of the product of many inputs using a tree-reduction, and the repeated squaring of a single input ciphertext, respectively. The given multiplication time is the average time spent per homomorphic multiplication. The parameter c is the length of the gadget vector used for relinearization, and t is the plaintext modulus.

Concretely, we compare our implementation of BGV with the BGV implementation of HElib [HS14], and our implementation of BFV with the BFV implementation of SEAL [23]. Despite the fact that we only compare implementations of the same scheme, there are already some differences between the libraries in the input and output format of the homomorphic multiplication. In particular, Fheanor stores elements in whatever representation they are currently available, HElib stores elements in double-RNS representation, and SEAL stores elements in single-RNS representation. Furthermore, HElib has a careful noise-management system, which dynamically performs modulus-switches during BGV whenever appropriate; while in Fheanor, the user is currently responsible for performing modulus-switches at the correct times. Therefore, to get an apples-to-apples comparison, we decide to benchmark circuits with multiple multiplications, and take the average time of all multiplications. The arithmetic circuits we consider

[6] This is commit 5dded27dc21eeb42d352ab91f6060326047cd114.

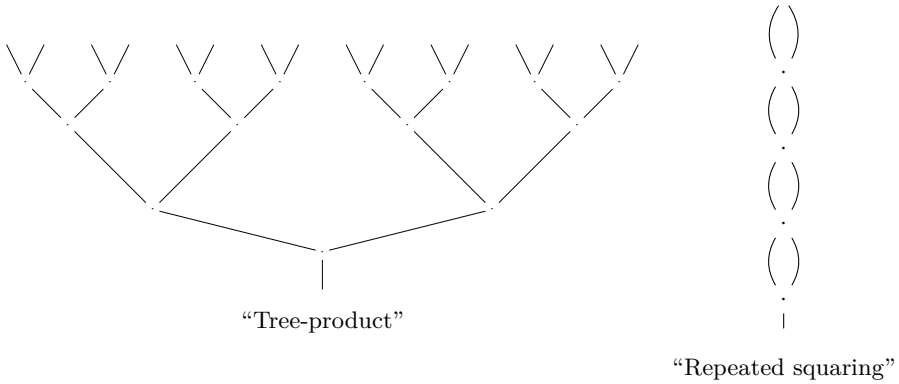

Fig. 3. The two circuits we use to benchmark homomorphic multiplications.

are shown in Fig. 3. The "Tree-product" circuit computes the product of 16 fresh input ciphertexts, using a binary-tree shaped circuit. The "Repeated squaring" circuit squares a single input ciphertext four times, so computing its 16^{th} power. We run each of these circuits using both BGV and BFV, and with various ring dimensions, where the ciphertext modulus q is always chosen to give 128 bits of security for the given ring dimension m according to the lattice estimator [APS15]. The standard deviation of the error distribution is fixed to $\sigma = 3.2$ and the secret is chosen from a uniform ternary distribution.

The timings of homomorphically evaluating these circuits under various parameters for BGV are displayed in Table 2, and for BFV in Table 3. For BGV, Table 2 shows that Fheanor can be almost $2\times$ faster in the power-of-two cyclotomic setting. In the non-power-of-two setting, the results show that while Fheanor is not quite as fast as HElib, the gap in performance is not very pronounced. In this setting, the Fheanor NTT (implemented using Bluestein FFT) is slower than the HElib NTT (implemented using a truncated FFT algorithm). We strongly believe that this gap could be closed if Fheanor were updated to include truncated and more heavily optimized FFT.

SEAL only implements BFV for power-of-two cyclotomics, so Table 3 focuses on this setting. In this case, the RNS conversions take more time than the NTTs and so represent the bottleneck in the computation. The RNS conversions in SEAL are very carefully optimized and, when compiled with `USE_INTRIN=ON`, use specialized CPU instructions for even better performance. Table 3 shows that SEAL noticeably outperforms Fheanor when compiled with `USE_INTRIN=ON` and slightly outperforms Fheanor otherwise. While further optimizations to the RNS conversion are likely to be included in further versions of Fheanor, we do not plan to include specific platform or CPU instruction optimizations.

4 Computing Trace and Norm

Within a slot $\mathbb{F}_{p^d}$ (or more generally a Galois Ring $\mathrm{GR}(p, e, d)$), we have the standard field trace and norm:

$$\mathrm{Tr}(\beta) := \sum_{i=0}^{d-1} \pi^i(\beta), \quad N(\beta) := \prod_{i=0}^{d-1} \pi^i(\beta)\,, \tag{3}$$

where π is the p^{th} power Frobenius automorphism[7]. Both trace and norm have numerous uses when performing homomorphic evaluations, which have been thoroughly explored in the power-of-two cyclotomic setting. For example, they are an important part of "ring switching" [GHPS13] and "ring tunnelling" [AP13], and can be used to speed up the linear transform during bootstrapping [CH18] in the power-of-two case or polynomial evaluations if the slots have large degree d [OPP23], as well as numerous other applications (e.g. [IIMP22]). The crucial point is that in the power-of-two case, the trace and norm can be computed using only $\log_2(d)$ key-switches, by the following recursive procedure:

$$\beta_0 := \beta, \quad \beta_{i+1} := \beta_i + \pi^{2^i}(\beta_i) \quad \Rightarrow \quad \mathrm{Tr}(\beta) = \beta_{\log_2(d)}\,.$$

It has been observed [CKL25] that a similar method applies in the non-power-of-two cyclotomic setting, assuming that the degree of the field extension $[\mathbb{F}_{p^d} : \mathbb{F}_p] = d$ is smooth. Indeed, if this is the case, the fundamental theorem of Galois theory and fact that $\mathrm{Gal}(\mathbb{F}_{p^d}/\mathbb{F}_p)$ is cyclic implies that there is a tower of intermediate fields

$$K_0 = \mathbb{F}_{p^d} \;/\; K_1 = \mathbb{F}_{p^{d_1}} \;/\; K_2 = \mathbb{F}_{p^{d_2}} \;/\; \ldots \;/\; K_r = \mathbb{F}_p\,,$$

where each degree $[K_i : K_{i+1}]$ is a prime factor of d. Hence, this allows us to compute the trace as shown in Algorithm 2.

In Algorithm 3 we present a novel, faster algorithm for this method based on addition chains, which we have implemented in Fheanor. An addition chain is a sequence of numbers $a_0 = 1, a_1, a_2, \ldots, a_l$ such that for each $i > 0$, there is $j, j' < i$ with $a_i = a_j + a_{j'}$. In general, given a fixed $a = a_l$, it is hard to find an addition chain of minimal length $l(a)$. However, we are only interested in the case that $a = d$ is a small number, so we can just store a precomputed table of shortest addition chains. Therefore, assume we have an addition chain $a_0, \ldots, a_l$ of length l with $a_l = d$. This then allows us to compute $\mathrm{Tr}(\beta)$ as shown in Algorithm 3.

Indeed, during the loop in Algorithm 3, we have the invariant

$$\beta_i = \sum_{j=0}^{a_i - 1} \pi^j(\beta)\,.$$

[7] In the case of a finite field $\mathbb{F}_{p^d}$, this is just the map $\beta \mapsto \beta^p$. In the case of a Galois ring $\mathrm{GR}(p, e, d)$ generated by a single generator θ over $\mathbb{Z}/p^e\mathbb{Z}$, it is the map $\sum a_i\theta^i \mapsto \sum a_i\tilde{\theta}^i$ where $\tilde{\theta}$ is the root of $\mathrm{MiPo}(\theta)$ that is congruent to θ^p modulo p.

Algorithm 2: The intermediate-field based method for computing the trace, using additions and Galois automorphisms [CKL25].

Input: Ring element $\beta \in \mathrm{GR}(p, e, d)$
Output: The trace $\mathrm{Tr}(\beta) \in \mathbb{Z}/p^e\mathbb{Z} \subseteq \mathrm{GR}(p, e, d)$

Compute the factorization of $d = p_1 \cdots p_r$ (with p_i not necessarily distinct)
Set $\beta_0 := \beta$
Set $d_0 := d$
for $i = 1$ *to* r **do**
    Set $d_i := d_{i-1}/p_i$
    Set $\beta_i := \sum_{j=0}^{p_i-1} \pi^{d_i j}(\beta_{i-1})$ (where π is the power-of-p Frobenius automorphism)
return $\mathrm{Tr}(\beta) = \beta_r$

Algorithm 3: Our new algorithm to compute the trace, using additions and Galois automorphisms.

Input: Ring element $\beta \in \mathrm{GR}(p, e, d)$; A short addition chain $a_0, ..., a_{l-1}, a_l = d$
Output: The trace $\mathrm{Tr}(\beta) \in \mathbb{Z}/p^e\mathbb{Z} \subseteq \mathrm{GR}(p, e, d)$

Set $\beta_0 := \beta$
for $i = 1$ *to* l **do**
    Choose $j, j' < i$ such that $a_i = a_j + a_{j'}$
    Set $\beta_i := \beta_j + \pi^{a_j}\beta_{j'}$ (where π is the power-of-p Frobenius automorphism)
return $\mathrm{Tr}(\beta) = \beta_l$

and thus $\beta_l = \mathrm{Tr}(\beta)$ as claimed. Clearly, this algorithm can also be adjusted to compute the field norm. The computational cost of this algorithm is given in Proposition 4.1.

Proposition 4.1. The field trace Tr in the field extension $\mathbb{F}_{p^d}/\mathbb{F}_p$ can be computed by a circuit containing $l(d)$ addition gates and $l(d)$ Galois gates, where $l(d)$ is the length of a shortest addition chain for d. Similarly, the field norm N in this field extension can be computed by a circuit containing $l(d)$ multiplication gates and $l(d)$ Galois gates.

If we use addition chains based on the factorization of d, it follows that this method is never worse than the intermediate-field based method of [CKL25]. Moreover, especially in cases where d has relatively large prime factors (in particular, d might be prime), this algorithm is significantly more efficient. To illustrate this, the number of automorphisms required to compute the trace with both methods is displayed in Table 4. Both approaches are also compared to a naive method that directly evaluates the trace.

Degree d	Number of required automorphisms		
	Naive	Algorithm 2 [CKL25]	Algorithm 3 (Ours)
$4 = 2^2$	3	2	2
$5 = 5$	4	4	3
$14 = 7 \cdot 2$	13	7	5
$15 = 3 \cdot 5$	14	6	5
$16 = 2^4$	15	4	4
$17 = 17$	16	16	5
$20 = 2^2 \cdot 5$	19	6	5
$21 = 3 \cdot 7$	20	8	6
$22 = 11 \cdot 2$	21	11	6
$40 = 2^3 \cdot 5$	39	7	6
$41 = 41$	40	40	7
$42 = 2 \cdot 3 \cdot 7$	41	9	7

Table 4. A comparison of the number of automorphisms required for computing the trace, using a naive algorithm, the intermediate-field method (Algorithm 2) of [CKL25], and our new addition-chain based method (Algorithm 3).

5 BFV and BGV Bootstrapping in Fheanor

Another important contribution of Fheanor is a new implementation of thin bootstrapping (see Fig. 2) for BFV and BGV. Our implementation follows the

general strategy described in Section 2.5. The individual components (linear transformation and digit extraction) are modelled in a generic way, which should make it easy to reuse them in other settings, in particular bootstrapping for variants of BFV or BGV.

Our implementation of the linear transformation follows the ideas of [Gee24] in the power-of-two case, and the ideas of [HS15] in the general case. There is more literature on digit extraction [CH18; GIKV23; MHWW24a; OPP23], which proposes optimizations that apply in different cases. Currently, only some of these techniques ([CH18] and parts of [GIKV23]) are implemented.

The general approach to digit extraction is to assume the plaintext modulus is a prime power p^e, and construct the floor division by p^v for $v < e$ using exact divisions by p and a polynomial $f \in \mathbb{Z}_{p^e}[X]$, which should satisfy

$$f(x) = \text{shortest-lift}(x \bmod p)\,.$$

The easiest way of computing $y = \lfloor x/p^v \rfloor$ using these would be

$$\begin{aligned}
x_0 &:= f(x)\,,\\
x_1 &:= f((x - x_0)/p)\,,\\
&\vdots\\
x_{v-1} &:= f((x - x_0 - px_1 - \ldots - p^{v-2}x_{v-2})/p^{v-2})\,,\\
x_v &:= (x - x_0 - px_1 - \ldots - p^{v-1}x_{v-1})/p^v\,.
\end{aligned}$$

However, this sequence of computations incurs a high multiplicative depth, which causes large noise growth. Hence, [HS15] proposed to instead compute

$$\begin{aligned}
x_{0i} &:= \Big(x - \sum_{j<i} p^j x_{(i-j)j}\Big)/p^i\,,\\
x_{(j+1)i} &:= g(x_{ji})\,,\\
y &:= \Big(x - \sum_{j<v} p^j x_{(e-j-1)j}\Big)/p^v\,,
\end{aligned}$$

where $g \in \mathbb{Z}_{p^e}[X]$ is a polynomial with the property

$$g(x + zp^i) \equiv x \bmod p^{i+1} \quad \text{for all } 1 \leq i < e,\ z \in \mathbb{Z}/p^e\mathbb{Z} \text{ and } x \in \{0, \ldots, p-1\}\,.$$

Variants of this have been proposed [CH18; OPP23; GIKV23; MHWW24a]. Except in the case $p = 2$, Fheanor currently only contains an implementation of the techniques from [CH18], which further reduce the multiplicative depth in the case that $r := e - v > 1$. More concretely, we use polynomials $f_k \in \mathbb{Z}_{p^e}[X]$ with the property

$$f_k \equiv \text{shortest-lift}(x \bmod p) \mod p^k\,.$$

These polynomials are then converted into arithmetic circuits using a low-depth variant of the Paterson-Stockmeyer evaluation scheme. However, we believe that

significant improvements to this method are possible, both in terms of the choice of polynomials, as well as during the conversion into an arithmetic circuit. Hence, we allow users to provide their own digit extraction circuits, and thus experiment with new methods for digit extraction.

5.1 New digit extraction circuits for $p = 2$

In the case $p = 2$, we extend the multivariate technique of [GIKV23] to compute optimal digit extraction circuits. More concretely, the authors propose four arithmetic circuits

$$\begin{aligned} &G_2(x) = (x_2),\ G_4(x) = (x_2, x_4),\ G_8(x) = (x_2, x_4, x_8), \\ &G_{16}(x) = (x_2, x_4, x_8, x_{16})\,, \end{aligned}$$

such that G_e has only $\log_2(e)$ multiplication gates, and the x_e satisfy

$$x_e \equiv \text{shortest-lift}(x \bmod p) \bmod p^e \quad \text{for } e \in \{2, 4, 8, 16\}\,.$$

This is the best possible case, as the function $f_e : \mathbb{Z}/2^e\mathbb{Z} \to \mathbb{Z}/2^e\mathbb{Z}$ that maps $x \mapsto \text{shortest-lift}(x \bmod 2)$ cannot be represented by a polynomial of degree less than e. The authors also note that there is no corresponding circuit G_{32} that only requires 5 multiplications. We remark that the circuit they give for $e = 16$ has a small mistake, and correct it.

However, the range $16 < e < 32$ is left open. We believe that this range is very relevant for practical parameters, since we require $v \approx \log_2(m)$ and $r > 1$, where $\log_2(m)$ is typically chosen as 15, 16, or 17 [HS15; CH18; OPP23; GIKV23]. Therefore, we tried to construct a suitable circuit $G_e(x) = (x_2, x_4, x_8, x_{16}, x_e)$ for the maximal possible e, and found that this is $e = 23$, yielding the circuit shown in Fig. 4.

$$\begin{aligned} G_2(x) &= x^2 \\ G_4(x) &= (G_2(x),\ G_2(x)^2) \\ G_8(x) &= (G_2(x),\ G_4(x),\ 112G_2(x) + (94G_2(x) + 121G_4(x))^2) \\ G_{16}(x) &= (G_2(x),\ G_4(x),\ G_8(x),\ 1984G_2(x) + 528G_4(x) + 22620G_8(x) + \\ &\quad (226G_4(x) + 113G_8(x))(8G_2(x) + 2G_4(x) + 301G_8(x))) \\ G_{23}(x) &= (G_2(x),\ G_4(x),\ G_8(x),\ G_{16}(x), \\ &\quad 4849408G_2(x) + 3564625G_4(x) + 2737008G_8(x) + 6563608G_{16}(x) + \\ &\quad (997183G_2(x) + 8295548G_4(x) + 419894G_8(x) + 879825G_{16}(x)) \cdot \\ &\quad (443729G_2(x) + 555132G_4(x) + 491350G_8(x) + 758385G_{16}(x))) \end{aligned}$$

Fig. 4. Our new optimal digit extraction circuit for $\mathbb{Z}/2^{23}\mathbb{Z}$.

In [GIKV23], the circuits were found by constructing the "generic" circuit whose constants are indeterminates of a multivariate polynomial ring, and then using Groebner basis techniques to find a suitable variable assignment. For $e = 23$, this approach did not suffice, since the ring $\mathbb{Z}/2^e\mathbb{Z}$ is not a field, and computing a Groebner basis over this ring seems prohibitively expensive. Instead, we first applied some algebraic manipulations to the polynomial system, and then encoded the system as a SAT formula, which we solved using the Varisat SAT solver [HAS19].

5.2 Benchmarks

We benchmark thin bootstrapping in Fheanor and HElib, on the same system as described in Section 3.4. The results are shown in Table 5. While the benchmarks in Section 3.4 compare implementations of almost the same algorithm (although with minor differences in modulus switching), this is not the case for bootstrapping. In particular, Fheanor uses the improved digit extraction circuits as described in Section 5.1 when $p = 2$. As a result, when the number of slots is small (so the linear transform takes less time), Fheanor is as fast as (or even faster than) HElib.

$(m, t, \log_2(q), c, \|s\|_0)$		HElib	Fheanor
$(37 \cdot 949, 4, 820, 10, 256)$	Total	98.2 s	92.2 s
	Slots-to-Coeffs	3.2 s	3.7 s
	Secret key product	0.7 s	0.9 s
	Coeffs-to-Slots	26.6 s	53.1 s
	Digit Extraction	67.7 s	34.5 s
$(337 \cdot 127, 4, 1120, 10, 256)$	Total	177.6 s	255.3 s
	Slots-to-Coeffs	6.3 s	6.3 s
	Secret key product	0.9 s	0.7 s
	Coeffs-to-Slots	49.8 s	160.0 s
	Digit Extraction	120.6 s	88.3 s

Table 5. Comparison of timings for BGV thin bootstrapping.

Acknowledgement We thank Seonhong Min for many fruitful discussions about the efficient implementation of various algorithms used by Fheanor. Simon Pohmann was supported by the EPSRC and the UK Government as part of the Centre for Doctoral Training in Cyber Security at Royal Holloway, University of London (EP/S021817/1). Parts of this research were conducted while Simon Pohmann was an invited researcher at KDDI Research, supported by International Exchange Program of National Institute of Information and Communications (NICT).

References

[ABB+22] A. Al Badawi, J. Bates, F. Bergamaschi, D. B. Cousins, S. Erabelli, N. Genise, S. Halevi, H. Hunt, A. Kim, Y. Lee, Z. Liu, D. Micciancio, I. Quah, Y. Polyakov, S. R.V., K. Rohloff, J. Saylor, D. Suponitsky, M. Triplett, V. Vaikuntanathan, and V. Zucca. "OpenFHE: Open-Source Fully Homomorphic Encryption Library". In: WAHC'22. ACM, 2022, pp. 53–63. DOI: 10.1145/3560827.3563379.

[APS15] M. R. Albrecht, R. Player, and S. Scott. In: *Journal of Mathematical Cryptology* 9.3 (2015), pp. 169–203. DOI: doi:10.1515/jmc-2015-0016.

[AP13] J. Alperin-Sheriff and C. Peikert. "Practical bootstrapping in quasilinear time". In: *Annual Cryptology Conference*. Springer. 2013, pp. 1–20.

[BEH+17] J.-C. Bajard, J. Eynard, M. A. Hasan, P. Martins, L. Sousa, and V. Zucca. "Efficient Reductions in Cyclotomic Rings - Application to Ring-LWE Based FHE Schemes". In: *SAC 2017: 24th Annual International Workshop on Selected Areas in Cryptography*. Vol. 10719. LNCS. 2017, pp. 151–171. DOI: 10.1007/978-3-319-72565-9_8.

[BEHZ16] J.-C. Bajard, J. Eynard, M. A. Hasan, and V. Zucca. "A Full RNS Variant of FV Like Somewhat Homomorphic Encryption Schemes". In: *SAC 2016: 23rd Annual International Workshop on Selected Areas in Cryptography*. Vol. 10532. LNCS. 2016, pp. 423–442. DOI: 10.1007/978-3-319-69453-5_23.

[Blu70] L. Bluestein. "A linear filtering approach to the computation of discrete Fourier transform". In: *IEEE Transactions on Audio and Electroacoustics* 18.4 (1970), pp. 451–455.

[BCH+25] M. Bolboceanu, A. Costache, E. Hales, R. Player, M. Rosca, and R. Titiu. "Designs for practical SHE schemes based on Ring-LWR". In: *IACR Communications in Cryptology* 2.1 (Apr. 8, 2025). ISSN: 3006-5496. DOI: 10.62056/av7tudy6b.

[BGV12] Z. Brakerski, C. Gentry, and V. Vaikuntanathan. "(Leveled) fully homomorphic encryption without bootstrapping". In: *Innovations in Theoretical Computer Science 2012, Cambridge, MA, USA, January 8-10, 2012*. ACM, 2012, pp. 309–325. DOI: 10.1145/2090236.2090262.

[CKL25] P. Chartier, M. Koskas, and M. Lemou. *Exploring General Cyclotomic Rings in Torus-Based Fully Homomorphic Encryption: Part I - Prime Power Instances*. Cryptology ePrint Archive, Paper 2025/488. 2025. URL: https://eprint.iacr.org/2025/488.

[CH18] H. Chen and K. Han. "Homomorphic Lower Digits Removal and Improved FHE Bootstrapping". In: *Advances in Cryptology – EUROCRYPT 2018, Part I*. Vol. 10820. LNCS. 2018, pp. 315–337. DOI: 10.1007/978-3-319-78381-9_12.

[CLPX18] H. Chen, K. Laine, R. Player, and Y. Xia. "High-Precision Arithmetic in Homomorphic Encryption". In: *Topics in Cryptology – CT-RSA 2018*. Vol. 10808. LNCS. 2018, pp. 116–136. DOI: 10.1007/978-3-319-76953-0_7.

[CCLS19] J. H. Cheon, H. Choe, D. Lee, and Y. Son. "Faster linear transformations in HElib, revisited". In: *IEEE Access* 7 (2019), pp. 50595–50604.

[CKKS17] J. H. Cheon, A. Kim, M. Kim, and Y. S. Song. "Homomorphic Encryption for Arithmetic of Approximate Numbers". In: *Advances in Cryptology – ASIACRYPT 2017, Part I*. Vol. 10624. LNCS. 2017, pp. 409–437. DOI: 10.1007/978-3-319-70694-8_15.

[CGGI16] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène. "Faster Fully Homomorphic Encryption: Bootstrapping in Less Than 0.1 Seconds". In: *Advances in Cryptology – ASIACRYPT 2016, Part I*. Vol. 10031. LNCS. 2016, pp. 3–33. DOI: 10.1007/978-3-662-53887-6_1.

[Con23] H. Contributors. *HEIR: Homomorphic Encryption Intermediate Representation*. https://github.com/google/heir. 2023.

[CP16] E. Crockett and C. Peikert. "$\Lambda \circ \lambda$: Functional Lattice Cryptography". In: *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. 2016, pp. 993–1005.

[DM15] L. Ducas and D. Micciancio. "FHEW: Bootstrapping Homomorphic Encryption in Less Than a Second". In: *Advances in Cryptology – EUROCRYPT 2015, Part I*. Vol. 9056. LNCS. 2015, pp. 617–640. DOI: 10.1007/978-3-662-46800-5_24.

[FV12] J. Fan and F. Vercauteren. *Somewhat Practical Fully Homomorphic Encryption*. Cryptology ePrint Archive, Report 2012/144. 2012. URL: https://eprint.iacr.org/2012/144.

[Gee24] R. Geelen. "Revisiting the Slot-to-Coefficient Transformation for BGV and BFV". In: *Communications in Cryptology* 1.3 (2024), p. 37. DOI: 10.62056/a01zogy4e-.

[GIKV23] R. Geelen, I. Iliashenko, J. Kang, and F. Vercauteren. "On Polynomial Functions Modulo p^e and Faster Bootstrapping for Homomorphic Encryption". In: *Advances in Cryptology – EUROCRYPT 2023, Part III*. Vol. 14006. LNCS. 2023, pp. 257–286. DOI: 10.1007/978-3-031-30620-4_9.

[GV23] R. Geelen and F. Vercauteren. "Bootstrapping for BGV and BFV Revisited". In: *Journal of Cryptology* 36.2 (2023), p. 12. ISSN: 1432-1378. DOI: 10.1007/s00145-023-09454-6.

[GV24] R. Geelen and F. Vercauteren. *Fully Homomorphic Encryption for Cyclotomic Prime Moduli*. Cryptology ePrint Archive, Paper 2024/1587. 2024. URL: https://eprint.iacr.org/2024/1587.

[Gen09] C. Gentry. "Fully homomorphic encryption using ideal lattices". In: *41st Annual ACM Symposium on Theory of Computing*. ACM Press, 2009, pp. 169–178. DOI: 10.1145/1536414.1536440.

[GHPS13] C. Gentry, S. Halevi, C. Peikert, and N. P. Smart. "Field switching in BGV-style homomorphic encryption". In: *Journal of Computer Security* 21.5 (2013), pp. 663–684.

[HPS19] S. Halevi, Y. Polyakov, and V. Shoup. "An Improved RNS Variant of the BFV Homomorphic Encryption Scheme". In: *Topics in Cryptology – CT-RSA 2019*. Vol. 11405. LNCS. 2019, pp. 83–105. DOI: 10.1007/978-3-030-12612-4_5.

[HS14] S. Halevi and V. Shoup. *Algorithms in HElib*. Cryptology ePrint Archive, Report 2014/106. 2014. URL: https://eprint.iacr.org/2014/106.

[HS15] S. Halevi and V. Shoup. "Bootstrapping for HElib". In: *Advances in Cryptology – EUROCRYPT 2015, Part I*. Vol. 9056. LNCS. 2015, pp. 641–670. DOI: 10.1007/978-3-662-46800-5_25.

[HS20] S. Halevi and V. Shoup. *Design and implementation of HElib: a homomorphic encryption library*. Cryptology ePrint Archive, Report 2020/1481. 2020. URL: https://eprint.iacr.org/2020/1481.

[HAS19] J. Harder, M. Augier, and C. Sauer. *Varisat v0.2.2*. https://github.com/jix/varisat. 2019.

[Har14] D. Harvey. "Faster arithmetic for number-theoretic transforms". In: *Journal of Symbolic Computation* 60 (2014), pp. 113–119.

[IIMP22] I. Iliashenko, M. Izabachène, A. Mertens, and H. V. Pereira. "Homomorphically counting elements with the same property". In: *Proceedings on Privacy Enhancing Technologies* (2022).

[BKS+21] F. Boemer, S. Kim, G. Seifu, F. D. de Souza, V. Gopal, et al. *Intel HEXL (release 1.2)*. https://github.com/intel/hexl. 2021.

[24] *Lattigo v6*. Online: https://github.com/tuneinsight/lattigo. EPFL-LDS, Tune Insight SA. 2024.

[LN16] P. Longa and M. Naehrig. "Speeding up the number theoretic transform for faster ideal lattice-based cryptography". In: *Cryptology and Network Security: 15th International Conference, CANS 2016, Milan, Italy, November 14-16, 2016, Proceedings 15*. Springer. 2016, pp. 124–139.

[LPR10] V. Lyubashevsky, C. Peikert, and O. Regev. "On Ideal Lattices and Learning with Errors over Rings". In: *Advances in Cryptology – EUROCRYPT 2010*. Vol. 6110. LNCS. 2010, pp. 1–23. DOI: 10.1007/978-3-642-13190-5_1.

[LPR13] V. Lyubashevsky, C. Peikert, and O. Regev. "A Toolkit for Ring-LWE Cryptography". In: *Advances in Cryptology – EUROCRYPT 2013*. Vol. 7881. LNCS. 2013, pp. 35–54. DOI: 10.1007/978-3-642-38348-9_3.

[MHWW24a] S. Ma, T. Huang, A. Wang, and X. Wang. "Accelerating BGV Bootstrapping for Large p Using Null Polynomials over $\mathbb{Z}_{p^e}$". In:

Advances in Cryptology – EUROCRYPT 2024, Part II. LNCS. 2024, pp. 403–432. DOI: 10.1007/978-3-031-58723-8_14.

[MHWW24b] S. Ma, T. Huang, A. Wang, and X. Wang. "Faster BGV Bootstrapping for Power-of-Two Cyclotomics Through Homomorphic NTT". In: *Advances in Cryptology – ASIACRYPT 2024, Part I*. LNCS. 2024, pp. 143–175. DOI: 10.1007/978-981-96-0875-1_5.

[OPP23] H. Okada, R. Player, and S. Pohmann. "Homomorphic Polynomial Evaluation Using Galois Structure and Applications to BFV Bootstrapping". In: *Advances in Cryptology – ASIACRYPT 2023, Part VI*. Vol. 14443. LNCS. 2023, pp. 69–100. DOI: 10.1007/978-981-99-8736-8_3.

[Reg05] O. Regev. "On lattices, learning with errors, random linear codes, and cryptography". In: *37th Annual ACM Symposium on Theory of Computing*. ACM Press, 2005, pp. 84–93. DOI: 10.1145/1060590.1060603.

[Sco17] M. Scott. "A note on the implementation of the number theoretic transform". In: *Cryptography and Coding: 16th IMA International Conference, IMACC 2017, Oxford, UK, December 12-14, 2017, Proceedings 16*. Springer. 2017, pp. 247–258.

[23] *Microsoft SEAL (release 4.1)*. https://github.com/Microsoft/SEAL. Microsoft Research, Redmond, WA. 2023.

[SSTX09] D. Stehlé, R. Steinfeld, K. Tanaka, and K. Xagawa. "Efficient Public Key Encryption Based on Ideal Lattices". In: *Advances in Cryptology – ASIACRYPT 2009*. Vol. 5912. LNCS. 2009, pp. 617–635. DOI: 10.1007/978-3-642-10366-7_36.

[Van04] J. Van der Hoeven. "The truncated Fourier transform and applications". In: *Proceedings of the 2004 international symposium on Symbolic and algebraic computation*. 2004, pp. 290–296.

[Vrb16] P. Vrbik. "An Illustrated Introduction to the Truncated Fourier Transform". In: *arXiv preprint arXiv:1602.04562* (2016).

[Zam22a] Zama. *Concrete: TFHE Compiler that converts python programs into FHE equivalent*. https://github.com/zama-ai/concrete. 2022.

[Zam22b] Zama. *TFHE-rs: A Pure Rust Implementation of the TFHE Scheme for Boolean and Integer Arithmetics Over Encrypted Data*. https://github.com/zama-ai/tfhe-rs. 2022.